

Especificación Técnica Definitiva y Matriz de Configuración YAML: Arquitectura de 10 Subagentes en Windows 11

1. Fundamentos de la Arquitectura Declarativa y Gobernanza de Agentes

La transición estratégica desde la programación imperativa hacia la configuración declarativa en ADK 1.12.0 marca el fin de la era de orquestación mediante scripts rígidos. En 2026, la estabilidad de los sistemas distribuidos de IA depende del cumplimiento estricto del esquema AgentConfig.json. Esta arquitectura permite desacoplar la lógica de control del motor de inferencia, garantizando que cada identidad, herramienta y límite normativo esté predefinido estáticamente. La importancia de este enfoque radica en la auditabilidad: un sistema configurado mediante YAML es resistente a la deriva de comportamiento, asegurando que los agentes operen bajo un contrato de ejecución predecible y robusto.

Análisis de Seguridad: Mitigación de Vulnerabilidades Industriales

La arquitectura de Antigravity 2.0 implementa una política de seguridad Deny-First, donde el acceso es nulo a menos que se declare explícitamente. Este modelo es vital para neutralizar vulnerabilidades críticas como GhostApproval (CVE-2026-12958). En este vector de ataque, repositorios maliciosos utilizaban enlaces simbólicos (symlinks) para desviar escrituras hacia archivos sensibles como ~/.ssh/authorized_keys. Antigravity elimina este riesgo mediante la resolución canónica de rutas físicas antes de cualquier interacción de aprobación, garantizando que el agente siempre opere sobre el archivo real y no sobre punteros ofuscados. Además, se impone un aislamiento que impide la ejecución de hooks o servidores MCP en directorios no marcados explícitamente como seguros por el usuario en ~/.gemini/config/.

2. Configuración del Entorno de Ejecución en Windows 11 y Gestión de Tokens

Para garantizar una orquestación multiactiva eficiente, es imperativo preparar el bloque de entorno del proceso padre en Windows 11. Una configuración deficiente del entorno impide la herencia transparente de credenciales, resultando en fallos de autenticación en subagentes críticos.

Protocolo de Inyección de Credenciales

Para habilitar el consumo industrial de API keys (GEMINI_API_KEY) y el funcionamiento de subagentes de automatización, siga este protocolo:

Declaración del Model Provider: Es obligatorio incluir "modelProvider": "gemini" en el archivo global ~/.gemini/antigravity-cli/settings.json. Sin esta clave, el arnés de Antigravity no procesará las variables de entorno para autenticación.

Resolución de la Variable HOME (Playwright/Chromium Fix): Los subagentes de navegación web fallan en Windows debido a la ausencia de la variable HOME. Ejecute el siguiente comando en PowerShell para mapear el perfil de usuario de forma permanente:

```
[Environment]::SetEnvironmentVariable("HOME", $env:USERPROFILE, "User")
```

Persistencia y Herencia: Tras el registro, reinicie el host de la terminal. Los subagentes heredarán automáticamente el bloque de entorno del proceso padre (parent process environment block), permitiendo el acceso a las credenciales sin re-autenticación.

Matriz de Servidores MCP (Aislamiento de Credenciales)

La gestión de conectores externos mediante el Protocolo de Contexto de Modelo se define en .agents/mcp_config.json:

Alcance

Ubicación (Windows)

Propósito

AuthProviderType

Global

%USERPROFILE%\gemini\config\mcp_config.json

Herramientas transversales (SaaS, DBs)

google_credentials

Proyecto

[REPO_ROOT]\.agents\mcp_config.json

Credenciales exclusivas del workspace

google_credentials

El uso de google_credentials permite al servidor MCP utilizar automáticamente las Application Default Credentials (ADC) del entorno Windows.

3. Arquitectura de Sidechains y Protección del Contexto Raíz

El mayor riesgo operativo en arquitecturas multiagente es el desperdicio termodinámico de tokens, donde la acumulación de trazas históricas degrada la atención del modelo y dispara los costos en un factor de 7x.

Mecanismo Técnico de Aislamiento

Antigravity 2.0 utiliza Sidechain Transcripts para delegar tareas sin contaminar el contexto principal. Cada subagente escribe su actividad en:

Archivos .jsonl: Registros secuenciales de eventos y llamadas a herramientas.

Archivos .meta.json: Almacenan el estado de la tarea y configuración del agente.

Al finalizar, el sistema ejecuta un "summary-only return": el orquestador principal solo recibe un resumen de alto nivel, preservando la ventana de contexto raíz para decisiones estratégicas.

Jerarquía de Compactación: Pre-model Shapers

Antes de cada llamada al LLM, el contexto atraviesa cinco moldeadores secuenciales:

Budget Reduction: Descarte de memoria obsoleta basado en límites de presupuesto.

Snip: Recorte de respuestas extensas de herramientas previas.

Microcompact: Eliminación de espacios en blanco y redundancias de formato.

Context Collapse: Proyección sintética no destructiva de la memoria.

Auto-Compact: Generación de resúmenes integrales al superar el umbral crítico de la ventana.

4. Matriz de Configuración YAML para los 10 Subagentes Especializados

Esta configuración instanciará un enjambre bajo el agente base antigravity-preview-05-2026, optimizado para Gemini 3.7 Flash.

1. DIRECTOR GENERAL: Orquestador raíz con visibilidad ejecutiva.

agent_class: LlmAgent

name: director_general

model: gemini-3.7-flash

description: Orquestador raíz. Gestiona la delegación estratégica y actualizaciones de subtrayectorias.

instruction: |

Usted es el Director General. Analice solicitudes y delegue tareas.

Implemente actualizaciones de subtrayectorias recursivas hacia la conversación raíz.

Mantenga la integridad del plan global sin saturar el contexto principal.

tools: [google_search, url_context, exit_loop]

disallowedTools: [execute_bash]

isolation_mode: in-process

permissionMode: default

2. ARQUITECTO WEB: Especialista en jerarquías visuales.

agent_class: LlmAgent

name: arquitecto_web

model: gemini-3.7-flash

description: Diseño semántico y maquetación de layouts de 2 columnas.

instruction: Diseñe estructuras HTML5 semánticas. Enforce jerarquías visuales de dos columnas y accesibilidad.

tools: [read_file, write_file]

isolation_mode: worktree

permissionMode: plan

3. DESARROLLADOR FRONTEND: Lógica de aplicación.

agent_class: LlmAgent

name: desarrollador_frontend

model: gemini-3.7-flash

description: Implementación de lógica JS, cotizadores y debounce.

instruction: Desarrolle lógica JS robusta. Implemente patrones de debounce y optimización de renderizado.

tools: [read_file, write_file]

isolation_mode: worktree

permissionMode: acceptEdits

4. GESTOR DE DATOS: Blindaje de persistencia.

agent_class: LlmAgent

name: gestor_datos

model: gemini-3.7-flash

description: Persistencia en localStorage y blindaje de integridad del carrito.

instruction: Implemente persistencia segura. Asegure que las operaciones en localStorage sean atómicas.

tools: [read_file, write_file]

isolation_mode: in-process

permissionMode: default

5. COMPILADOR TAILWIND: Ejecución nativa.

agent_class: LlmAgent

name: compilador_tailwind

model: gemini-3.7-flash

description: Compilación nativa de estilos CSS en Windows 11.

instruction: Ejecute 'npx.cmd tailwindcss' para compilar. Verifique el output de consola para errores sintácticos.

tools: [run_command]

disallowedTools: [web_fetch]

isolation_mode: in-process

permissionMode: acceptEdits

6. OPTIMIZADOR MOBILE UX: Ergonomía táctil.

agent_class: LlmAgent

name: optimizador_mobile_ux

model: gemini-3.7-flash

description: Optimización para dispositivos móviles y carrusel snap-x.

instruction: Refactorice para ergonomía móvil. Implemente contenedores con desplazamiento 'snap-x mandatory'.

tools: [read_file, write_file]

isolation_mode: worktree

permissionMode: plan

7. SEGURIDAD Y BACKUPS: Resiliencia de archivos.

agent_class: LlmAgent

name: seguridad_backups

model: gemini-3.7-flash

description: Generación obligatoria de backups pre-escritura.

instruction: |

Procedimiento crítico: Genere archivos .bak antes de cualquier operación de escritura.

Valide la integridad física del backup antes de confirmar la tarea.

tools: [read_file, write_file, run_command]

isolation_mode: in-process

permissionMode: auto

8. AUDITOR QA: Inspección técnica.

agent_class: LlmAgent

name: auditor_qa

model: gemini-3.7-flash

description: Validación de código y verificación de Exit Code 0.

instruction: Inspeccione clases CSS y lógica. Todo proceso de herramienta debe retornar 'Exit Code 0'.

tools: [run_command, read_file, run_tests]

isolation_mode: remote
permissionMode: default

9. INGENIERO PYTHON: Refactorización aislada.

agent_class: LlmAgent
name: ingeniero_python
model: gemini-3.7-flash
description: Refactorización masiva de backend con aislamiento físico.
instruction: Refactorice lógica de backend siguiendo estándares PEP8 de alta densidad.
Use worktrees aislados.
tools: [read_file, write_file, run_tests]
isolation_mode: worktree
permissionMode: acceptEdits

10. GESTOR GIT: Integridad de versiones.

agent_class: LlmAgent
name: gestor_git
model: gemini-3.7-flash
description: Control de versiones y commits atómicos.
instruction: Gestione el historial. Asegure commits atómicos con mensajes descriptivos
bajo norma técnica.
tools: [run_command]
isolation_mode: in-process
permissionMode: acceptEdits

5. Protocolo de Purga y Transición de Arquitectura

Para implementar el modelo declarativo 2026, se deben eliminar todas las configuraciones residuales de arquitecturas obsoletas.

Guía de Limpieza (PowerShell)

Ejecute estos comandos para sanear el entorno de Windows:

Eliminar configuraciones antiguas:

```
Remove-Item -Recurse -Force "$HOME\.gemini\"
```

Redireccionar habilidades (Skills):

```
if (!(Test-Path ".agents\skills")) { New-Item -ItemType Directory -Force ".agents\skills" }
```

Mover archivos .md de habilidades a la nueva ruta

Validación de Integridad

Verifique la activación de la arquitectura mediante la CLI de Antigravity (agy):

agy /config: Confirma que el modelProvider y agentMode son correctos.

agy /usage: Monitorea el consumo de tokens en paralelo para evitar el agotamiento de la cuota.

6. Conclusiones y Gobernanza Continua

La escalabilidad del sistema reside en la consistencia entre el archivo YAML y el esquema central. Esta arquitectura garantiza que el desarrollo de software autónomo sea seguro, auditable y térmicamente eficiente.

Recomendaciones Finales:

Auditoría de Cuotas: Revise /usage tras cada ejecución de enjambre; la ejecución paralela puede agotar cuotas Pro en pocos ciclos.

Priorización de Aislamiento: Use siempre isolation_mode: worktree para tareas de desarrollo masivo para evitar colisiones en el espacio de trabajo.

Actualización de Modelos: Mantenga la referencia a la serie Gemini 3.7 o superior para maximizar la eficiencia de los moldeadores de contexto.

Este sistema representa la robustez industrial necesaria para la ingeniería de software en la era de los agentes autónomos.